Maximum Area Rectangle from Given Points

Company: Google
Round: Interview Round
Year: 2026
Difficulty: Medium
Topic: Arrays, HashSet, Geometry
Source: https://www.desiqna.in/19213/google-interview-experiences-sde-2026-set-74

Problem Description

Given a set of N points on a 2D plane, where each point is represented as [x, y], find the maximum possible area of a rectangle that can be formed using the given points as its four corners.

A point can be used only if it exists in the given set.

The goal is to find the rectangle with the largest area. If no valid rectangle can be formed, return 0.

Input Format

The first line contains N, the number of points.

The next N lines contain two space-separated integers x and y, representing the coordinates of each point.

Output Format

Print a single integer representing the maximum area of a rectangle that can be formed using the given points.

If no rectangle can be formed, print 0.

Sample Input

6

1 1

1 4

5 1

5 4

2 2

3 3

Sample Output

12

Explanation

The points (1,1), (1,4), (5,1), and (5,4) make a rectangle.
The distance between x = 1 and x = 5 is 4, so the length is 4.
The distance between y = 1 and y = 4 is 3, so the width is 3.
Therefore, the area is 4 × 3 = 12.
No other points can make a bigger rectangle.
So, the maximum area is 12.

Approach to Solve This Problem:

1. Store all points in a HashSet:
   - Put every given point into a HashSet.
   - This allows us to quickly check whether a particular point exists.
   - For example, if we have (1,1), we can quickly check whether (1,4) exists.
2. Choose two points as opposite corners:
   - Take every possible pair of points (x1, y1) and (x2, y2).
   - These two points can be considered as the diagonal corners of a rectangle.
   - They must have different x and y coordinates.
3. Find the other two corners:
   - If (x1,y1) and (x2,y2) are opposite corners, the other two corners must be:
     - (x1,y2)
     - (x2,y1)
   - Check whether both of these points are present in the HashSet.
4. Calculate the area:
   - If both required points exist, a valid rectangle has been formed.
   - Calculate:
     width = |x2 - x1|
     height = |y2 - y1|
   - Then calculate the area as width × height.
   - Keep updating the maximum area found.
5. Final result:
   - Continue checking all possible pairs of points.
   - The largest valid area found is the answer.
   - If no four points form a rectangle, return 0.

Java Solution:

Main.java > Main

```java
import java.util.*;

public class Main {

    public static int maxArea(int[][] points) {

        int n = points.length;
        int maxArea = 0;

        // Store all points for quick lookup
        HashSet<String> set = new HashSet<>();

        for (int[] point : points) {
            set.add(point[0] + "," + point[1]);
        }

        // Choose two points as opposite corners
        for (int i = 0; i < n; i++) {

            int x1 = points[i][0];
            int y1 = points[i][1];

            for (int j = i + 1; j < n; j++) {

                int x2 = points[j][0];
                int y2 = points[j][1];

                // They must have different x and y coordinates
                if (x1 == x2 || y1 == y2) {
                    continue;
                }

                // Find the other two corners
                String point3 = x1 + "," + y2;
                String point4 = x2 + "," + y1;

                // Check if both points exist
                if (set.contains(point3) && set.contains(point4)) {

                    int width = Math.abs(x2 - x1);
                    int height = Math.abs(y2 - y1);

                    int area = width * height;

                    maxArea = Math.max(maxArea, area);
                }
            }
        }

        return maxArea;
    }

    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[][] points = new int[n][2];

        for (int i = 0; i < n; i++) {
            points[i][0] = sc.nextInt();
            points[i][1] = sc.nextInt();
        }

        System.out.println(maxArea(points));

        sc.close();
    }
}
```

Solution:

1)Initialize variables:

n stores the total number of points.

maxArea stores the largest rectangle area found so far.

Initially, maxArea = 0.

2) Store all points in a HashSet:

We store every point in the HashSet.

This allows us to quickly check whether a particular point exists.

3) Select two points:

Use two loops to select every possible pair of points.

Consider the selected two points as opposite corners of a possible rectangle.

4)Check their coordinates:

If both points have the same x or the same y, they cannot be opposite corners.

So, we skip that pair.

6) Find the remaining two corners:

If the selected points are (x1, y1) and (x2, y2), the other corners must be:

(x1, y2)

(x2, y1)

7) Check whether the other corners exist:

Use HashSet.contains() to check both points.

If both exist, all four points form a valid rectangle.

8) Calculate the area:

Calculate the width using |x2 - x1|.

Calculate the height using |y2 - y1|.

Multiply them to get the rectangle's area.

9) Update the maximum area:

Compare the current area with maxArea.

Keep whichever value is larger.

10) Return the answer:

After checking all possible pairs, return maxArea.

If no rectangle is possible, maxArea remains 0.

Time complexity :- $O(N^2)$.

Space Complexity :- O(N).